Code Brawl

Production Engineering Roadmap

August 2026

Contents

A single source of truth for taking Code Brawl from its current authenticated-backend foundation to a serious, production-quality competitive coding platform.

How to read this document. It is organized by **dependency stages**, not by time. Finish a stage's "definition of done" before starting the next where the arrow (↓) implies a hard dependency. Within each major system you will see three maturity tiers — **MVP → Solid → Production-grade** — so you always know what is "good enough for now" versus what is "the real thing later." Nothing here contains implementation code by design: the tasks tell you *what* to build and *why*, and point you at the exact concepts to learn before you write each piece yourself.

---

1. Project Overview

Code Brawl is a competitive programming platform combining solo problem-solving with real-time 1v1 (and eventually multiplayer) coding battles, backed by a rating and leaderboard system. The core user journey is:

```
Choose a problem → Understand it → Write a solution → Submit code
→ Code is compiled & executed against test cases → Receive a verdict
→ Compete against other users → Earn/adjust rating → Climb the leaderboard
```

The technically defining characteristic of this project — the thing that separates it from a generic CRUD portfolio app — is that **it executes untrusted, user-submitted code and must do so safely, deterministically, and at bursty scale.** Almost every hard problem in the project

(sandboxing, async job processing, worker fleets, resource limits, concurrency, real-time state, fair rating math) radiates out from that single requirement. Treat the judge as the heart of the system; everything else is supporting infrastructure around it.

A useful mental model of the three concentric problems you are actually solving:

Layer	What it really is	Why it's hard
Content platform	Problems, test cases, users, submissions, history	Data modeling, integrity, pagination, search — solvable with solid fundamentals
Judge	Compile + run arbitrary code under strict limits, return a verdict	Security isolation + async orchestration + determinism — genuinely hard
Competition	Real-time battles, matchmaking, rating, leaderboards	Distributed state + concurrency + fairness — hard at scale

---

2. Honest Project Rating

Ratings are out of 10, judged against a **serious backend built by one learning developer**, not against a funded team. I've separated *learning/portfolio value* from *product value* because they diverge sharply here.

Dimension	Score	Honest justification
Technical difficulty	8.5	Untrusted code execution + async judging + real-time competition is legitimately hard. This is not a to-do app.
Backend depth	9	Touches nearly every serious backend topic: security, concurrency, queues, workers, caching, transactions, real-time. Excellent surface area.
Database complexity	7	Meaningful (submission history at scale, leaderboards, concurrent stat updates, integrity under races) but not extreme. No sharding/geo-distribution needed for a long time.
Security challenges	9.5	Executing hostile code is one of the hardest security problems in all of web engineering. This alone elevates the project.
Scalability potential	8	Real bursty load (contest spikes), stateful real-time, a fleet that must scale independently. Rich scaling story.
Real-world usefulness	6	The problem domain is real, but the space is saturated (LeetCode, Codeforces, HackerRank, CodinGame). Useful, not novel.
Portfolio value	9.5	If the judge and concurrency are done properly, this is a *standout* portfolio piece that signals senior-level concerns. Most bootcamp projects never touch this.
Learning value	10	Close to ideal as a teaching vehicle. It forces you into the exact topics that make a backend engineer, and punishes shortcuts.
Potential to become a real product	4.5	Crowded market, no obvious differentiator yet, high infra cost (running code costs money), unclear monetization. Possible with a sharp niche, but that's a business problem, not an engineering one.

Weighted overall: ~8.3/10 as a learning + portfolio project; ~4.5/10 as a commercial product.

Blunt honesty:

- **Your ceiling is learning/portfolio, and that's a great ceiling.** Build it to be the best possible demonstration of production backend skill. Do not quit your day job over it, and don't let "will this be a business" pressure distort technical decisions.
- **The single biggest risk to this project is scope, not difficulty.** Your `ReadMe.md` and feature list already mention notifications, friends system, OAuth2, contests, email verification, moderation, audit logs, AI (there are commented-out Spring AI deps in your `pom.xml`), gRPC, and Spring Shell. Most of these are distractions right now. The project will *feel* stuck if you build breadth instead of depth. Depth here means: **problems → real judge → battles → rating**, done well, before anything else.

- **Do not build your own sandbox from scratch as your first execution attempt.** Secure isolation of hostile code is a research-grade problem. The senior move is to stand on battle-tested isolation (a dedicated engine like Judge0, or the IOI `isolate` tool, running inside containers) and spend your effort on the *orchestration* around it. More on this in §9 and §14.

---

3. Current-State Assessment

I read your actual source, not just your feature list. Overall: **the foundation is genuinely good for someone learning** — you have a real layered structure, DTOs, a global exception handler, a custom `UserDetails`, refresh-token persistence with revocation, and even a `learn.md` where you documented the Spring Security request flow. That last detail matters: you're learning deliberately, not copy-pasting. Keep doing that.

But there are concrete correctness, integrity, and hygiene issues that will bite you later. I'm listing them precisely because you asked me to challenge you.

3.1 What you've built well

- Clear layered separation (`Controller` / `Security`(service) / `Repo` / `Entity` / `Dtos` / `Mapper` / `ExceptionHandling` / `ApiResponses`).
- A consistent success envelope (`Response<T>`) and a `GlobalExceptionHandler` with typed exceptions — this is more discipline than most beginners show.
- Refresh tokens persisted in `refresh_tokens` with a `revoked` flag and expiry — the right shape for session/device management.
- `CustomUserDetails` correctly wraps the entity and exposes the password hash; BCrypt is configured.
- Stateless `SessionCreationPolicy.STATELESS` + a `OncePerRequestFilter` JWT filter — correct backbone.

3.2 Bugs & correctness issues (fix these — they are real)

#	Issue	Where	Why it matters
B1	**Double `ROLE_` prefix.** `RoleEnum` values are `ROLE_USER`/`ROLE_ADMIN`/..., and `CustomUserDetails.getAuthorities()` returns `"ROLE_" + role.name()` → authority becomes `ROLE_ROLE_ADMIN`.	`CustomUserDetails`, `RoleEnum`	The moment you write `@PreAuthorize("hasRole('ADMIN')")` (which checks for `ROLE_ADMIN`), it will **silently deny** every admin. Your entire RBAC foundation is currently mis-wired. Decide on one convention: store bare `USER`/`ADMIN` in the enum and let `hasRole` add the prefix, *or* store `ROLE_*` and use `hasAuthority`. Not both.
B2	**`username` and `email` are not `UNIQUE` at the DB level.** They're `nullable=false` with **non-unique** indexes (`idx_users_email`, `idx_users_username`).	`UserEntity`	Uniqueness is only enforced by `existsByUsername/Email` checks in the service, which is a **race condition**: two concurrent signups both pass the check, both insert. You will get duplicate accounts. Integrity must be enforced by the database, not application code.
B3	**`UserEntity.getAuthorities()` returns `List.of()`** (empty) while `CustomUserDetails` returns real authorities.	`UserEntity`	`UserEntity` implements `UserDetails` *and* you have a separate `CustomUserDetails`. Two `UserDetails` implementations for one concept is confusing and a latent bug source. Pick one (I recommend the wrapper `CustomUserDetails` and *not* having the entity implement `UserDetails`).

#	Issue	Where	Why it matters
B4	**Token lifetimes are debug values.** Access token = **40 seconds**, refresh = **2 minutes** in `AuthUtil`, but the login cookie sets refresh `maxAge` = **7 days** and the service stores `expiresAt = now + 7 days`.	`AuthUtil`, `LoginController`, `AuthService`	The JWT's own expiry (2 min) and the persisted/cookie expiry (7 days) disagree — refresh will fail after 2 minutes regardless of the 7-day cookie. Pick real values (e.g., access ~15 min, refresh ~7–30 days) and make the three sources agree.
B5	**Access & refresh tokens share one secret.** `getAccessTokenSecretKey()` and `getRefreshTokenSecretKey()` return the same key.	`AuthUtil`	If the access-token secret ever leaks (they're handed out constantly), refresh tokens are compromised too. Use two distinct secrets.
B6	**`SubmissionEntity.battle` is `nullable=false`** and there is **no direct `problem` link** on a submission.	`SubmissionEntity`	This makes battles mandatory for *every* submission and blocks solo/practice submissions entirely — yet your own user journey starts with solo problem solving. A submission's primary relationship should be to a **problem** (+ optional battle). This is a modeling error that will force a painful migration later.
B7	**`ProblemEntity` has a single `language` field.**	`ProblemEntity`	A problem is language-agnostic; the *submission* has a language. Putting language on the problem conflates the two. Also missing: statement/description, slug/title, examples, **test cases (no `TestCaseEntity` exists at all)**, time/memory limits, tags, status (DRAFT/PUBLISHED), author. As-is, the problem can't actually be judged.
B8	**You log secrets.** The JWT filter does `log.info("Authorization Header: {}", authHeader)` and logs usernames/subjects at INFO.	`JwtAuthenticationFilter`	Never log tokens or credentials. This leaks bearer tokens into log files/aggregators. Remove before anything approaches production, and lower auth logs to DEBUG.

3.3 Configuration & hygiene issues

#	Issue	Why it matters
C1	**Secrets committed to git.** `application.properties` contains a real DB password (`@Bimshuthedeveloper_0562`), the JWT secret, and a default `spring.security.user.password=pass`.	These are in your git history now. **Rotate that DB password**, move all secrets to environment variables / externalized config, and add a `.env`-style ignored file. This is the highest-priority hygiene fix.
C2	**`spring.jpa.hibernate.ddl-auto=update`.**	Hibernate is silently managing your schema. `update` never drops/renames, can't be reviewed or rolled back, and drifts between environments. You must move to **versioned migrations (Flyway or Liquibase)** before the schema gets complex — which is *now*.
C3	**`compose.yaml` and `application.properties` disagree** (DB name `mydatabase`/user `myuser` vs `codebrawl`/`postgres`) and the compose port isn't bound to a host port.	Your local Postgres story is inconsistent; standardize it so tests and app point at the same reproducible DB.
C4	**Heavy unused dependencies:** `spring-grpc-server-web`, `spring-shell-starter`, and commented-out `spring-ai` (Anthropic/Azure).	These add build weight, attack surface, and confusion, and none are used. Remove them until you have a concrete reason. (gRPC *may* later be justified for orchestrator↔worker comms — but not now.)
C5	**No migrations, no pagination, no custom queries, no integration tests.** `grep` finds no Flyway/Liquibase, no `Pageable`, no `@Query`, no Testcontainers; the only test is the default context-load test.	This is the real gap. The cross-cutting foundation (migrations, pagination conventions, integration testing with a real Postgres) should exist *before* you add feature entities, because retrofitting them is expensive.

#	Issue	Why it matters
C6	**API shape inconsistencies.** `/refresh` returns a raw `String` while everything else returns `Response<T>`; `LoginController` is `@Controller` while `SignupController` is `@RestController`; `Error<T>` and `Response<T>` are two different envelopes.	Frontend integration later depends on *one* predictable response contract. Standardize now while it's cheap (see §4 and Frontend notes).

3.4 Package/structure observations

- Enums are scattered: `DifficultyEnum` and `LanguageEnum` live in the **root package**, `RoleEnum` under `Entity`, `Verdict` under `Entity.Model`. Pick a consistent home (ideally per-feature — see §4).
- You're organized **package-by-layer**. That's fine today but starts to hurt as features multiply. §4 explains the move to **package-by-feature** and why.
- `Controller/TempTask.java` is a leftover test endpoint — delete it.

Bottom line: you don't need to rebuild auth (you asked me not to, and you shouldn't). But before building new features, spend a focused effort on a **foundation-hardening pass** (§5, Stage 0) that fixes B1–B8/C1–C6. These are cheap now and expensive later.

---

4. Recommended Architecture

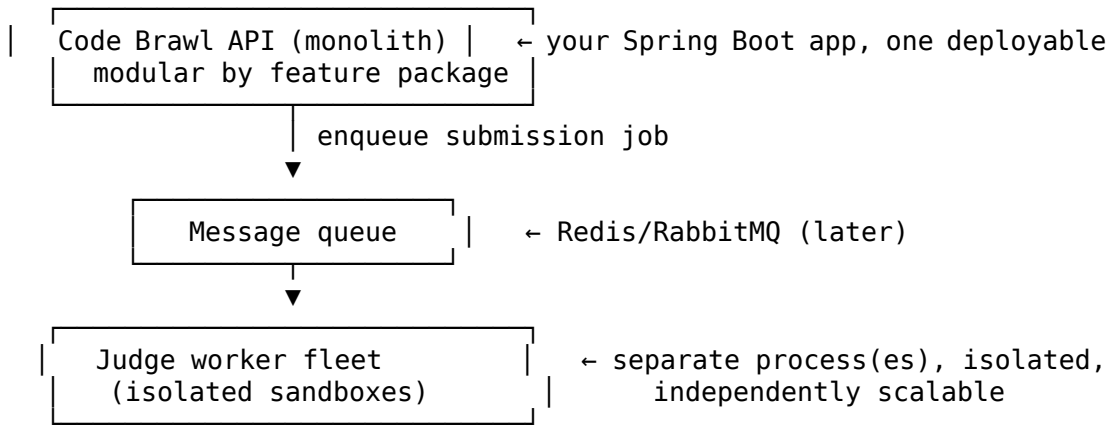
4.1 Monolith vs modular monolith vs microservices — the actual recommendation

Build a modular monolith with a separate worker fleet for code execution. Do not build microservices.

Here is the reasoning, because you asked to understand *why*, not just *what*:

- **Microservices solve an organizational and independent-scaling problem you do not have.** They cost you distributed transactions, network failure modes, service discovery, cross-service observability, and deployment complexity — all before you've shipped a single problem. For a solo developer learning the domain, microservices are pure tax.
- **A plain layered monolith** (what you have) is fine now but tends to rot into a "big ball of mud" where every service can reach every repository, and feature boundaries blur.
- **A modular monolith** keeps the operational simplicity of one deployable while enforcing **feature module boundaries** in code. You get 90% of the architectural discipline of microservices with 10% of the pain, and — crucially — if one module ever genuinely needs to become its own service, clean boundaries make that extraction possible instead of mythical.

The one true exception: the **code-execution worker** should be a **separate process/service from day one of building the judge.** Not because microservices are cool, but because it has fundamentally different requirements: it runs hostile code, must be isolated, must scale independently under contest spikes, and must be able to die/restart without touching your API. So the real topology is:



4.2 Package-by-feature (move to this deliberately)

Refactor from package-by-layer (`Controller/`, `Service/`, `Repo/`…) to **package-by-feature**:

```
com.codebrawl
├── shared/        (cross-cutting: ApiResponse, exceptions, config, security primitives)
├── user/          (UserEntity, ProfileEntity, UserService, UserController, UserRepo, dtos)
├── auth/              (existing security/JWT code lives here)
├── problem/       (ProblemEntity, TestCaseEntity, TagEntity, admin + public controllers)
├── submission/        (SubmissionEntity, lifecycle, queue producer)
├── judge/             (worker contract, verdict model, result consumer)
├── battle/            (BattleEntity, matchmaking, state machine)
├── rating/            (rating math, leaderboard queries)
└── notification/      (later)
```

Why this matters: a new engineer (or you in six months) can understand one feature by opening one package. It also makes module boundaries *visible* — if `battle` starts importing `problem`'s repository directly instead of going through a `problem` service, that's a smell you can now see. Keep each feature's entities/DTOs/services/controllers together.

4.3 Layer responsibilities (the contract)

You said you don't want to "blindly create controllers, services, repositories." Here's the discipline that separates production code from tutorial code:

Layer	**Does**	**Must NOT do**
Controller	HTTP concerns only: map request → DTO, validate input shape, call *one* service method, map result → response DTO, set status codes.	Contain business logic, touch repositories, manage transactions, or ever expose an entity.
Service	The business logic + the **transaction boundary** (`@Transactional` lives here). Orchestrates repositories, enforces invariants, performs domain rules.	Know about HTTP (no `HttpServletRequest`, no `ResponseEntity`), or leak persistence details upward.
Repository	Data access only: queries, persistence.	Contain business rules. Return entities (internally) or projections (for reads).
Domain / mapper	Convert entity ↔ DTO (MapStruct, which you already use), hold pure domain calculations (e.g., rating math) as testable units.	Depend on Spring web/persistence infrastructure where avoidable.

Key rules of thumb:

- **Validation happens in two places, deliberately:** *syntactic* validation (not null, length, email format, range) via Bean Validation (`@Valid` on the request DTO at the controller edge — you already do this); *semantic/business* validation (e.g., "this problem is published", "this user isn't already in a battle") in the **service**, because only the service has the data to decide.
- **Transactions belong on service methods, never on controllers or repositories.** One business operation = one transaction. This is where you'll later reason about isolation levels and locking (§7).
- **Never expose entities over HTTP.** Always map to DTOs. This prevents lazy-loading serialization blow-ups, avoids leaking internal fields (password hash!), and decouples your API contract from your schema so you can refactor the DB without breaking clients. You already do this for auth — make it a universal rule.

4.4 API contract decisions to lock in now (so the frontend is painless later)

You haven't built the frontend, and you shouldn't let that block you — but a few decisions made now make integration trivial later, and are expensive to change once clients depend on them:

- **One response envelope, everywhere.** Right now `Response<T>`, `Error<T>`, and raw strings coexist. Pick one success shape and one error shape and use them without exception. Strongly consider Spring's built-in **`ProblemDetail` (RFC 7807)** for errors — it's a standard the frontend and tooling already understand, instead of a bespoke envelope.

- **Version the API** with a `/api/v1` prefix from the start. Cheap now, saves you from breaking clients later.
- **Consistent pagination contract**: decide the response shape for paged lists (content + page metadata) once, and reuse it for problems, submissions, leaderboards.
- **Stable enums as strings** (you already use `EnumType.STRING`) and stable field names — the frontend will bind to these.
- **CORS as configuration**, not code sprinkled around — you'll need it the moment a browser app calls the API from another origin (§14).
- **OpenAPI/Swagger** (springdoc) generates a live contract the frontend can consume and even code-generate a client from. Add it early; it doubles as free documentation.

---

5. Development Stages (Correct Order)

Organized by dependency, not time. The arrow means "the later stage genuinely depends on the earlier one." Within a stage, the checklist is the **definition of done**, and "Verify" tells you how to prove it's actually done.

```
Stage 0  Foundation Hardening        (fix what exists; add cross-cutting bones)
   ↓
Stage 1  Problem Domain              (the content everything else needs)
   ↓
Stage 2  Submission Lifecycle      (persist + state machine, execution still stubbed)
   ↓
Stage 3  The Judge (real execution)  (isolated worker, async, verdicts)  ← the hard core
   ↓
Stage 4  Solo Practice Complete      (history, stats, search, pagination polished)
   ↓
Stage 5  Battles (async first)       (matchmaking, battle state, scoring)
   ↓
Stage 6  Rating & Leaderboards       (Elo/Glicko, ranking queries)
   ↓
Stage 7  Real-Time Layer             (WebSockets, live battle UX)
   ↓
Stage 8  Production Hardening      (observability, scale, ops) — partly continuous
```

> **Why this order?** You cannot judge submissions without problems + test cases (Stage 1 → 2/3). Battles are just "a submission race against a shared problem," so they need a working judge first (Stage 3 → 5). Rating is a function of battle outcomes (Stage 5 → 6). Real-time is a UX layer on top of battles that already work correctly via request/response, so it comes *after* battles are correct (Stage 5 → 7) — building real-time first is the classic trap that produces a flashy demo sitting on a broken core.

Stage 0 — Foundation Hardening *(do this first, it's mostly small)*

Goal: make the existing codebase trustworthy and add the cross-cutting infrastructure that all later features assume. This is **not** rebuilding auth — it's fixing specific defects and laying bones.

- ☐ Fix B1 (double `ROLE_`), B2 (unique constraints on `username`/`email` — at the DB level), B3 (one `UserDetails`), B4 (real token lifetimes, made consistent), B5 (separate secrets), B8 (stop logging tokens).
- ☐ C1: move all secrets to environment variables / externalized config; **rotate the leaked DB password**; keep secrets out of git.
- ☐ C2: introduce **Flyway** (or Liquibase). Switch `ddl-auto` to `validate`. Write your first migration to match the current schema, then never let Hibernate manage schema again.
- ☐ C4: remove unused gRPC / Spring Shell / Spring AI dependencies.
- ☐ C6: standardize on **one response envelope** + `/api/v1` prefix; decide on `ProblemDetail` for errors.
- ☐ Add **springdoc-openapi** (Swagger UI) so every endpoint is documented from now on.
- ☐ Add **Spring Boot Actuator** (health/info) and structured logging with a request correlation id.

- ☐ Stand up **Testcontainers** with real PostgreSQL and write one repository integration test to prove the harness works. (Not H2 — see §13 for why.)
- ☐ Delete `TempTask`.

Verify: app boots with `ddl-auto=validate` against a Flyway-migrated schema; a duplicate-username signup now fails at the DB; `@PreAuthorize("hasRole('ADMIN')")` on a throwaway endpoint actually admits an admin and rejects a user; no secret appears in the repo; Swagger UI lists your endpoints; one Testcontainers test is green.

Stage 1 — Problem Domain

Goal: problems that can actually be judged, plus admin authoring and public browsing.

- ☐ Model `ProblemEntity` properly (title, slug, statement/markdown, difficulty, time/memory limits, status DRAFT/PUBLISHED, author, timestamps) — **remove `language` from the problem** (B7).
- ☐ Add `TestCaseEntity` (input, expected output, `isSample`/hidden flag, ordering, weight) related many-to-one to problem.
- ☐ Add `TagEntity` + many-to-many with problems (categories).
- ☐ Public read APIs: list problems with **pagination + filtering (difficulty, tag, search) + sorting**; get one published problem (samples only, never hidden test cases).
- ☐ Admin APIs (RBAC-gated): create/edit/publish problems, manage test cases.

Verify: an anonymous/user role can list & view *published* problems and their *sample* tests but **cannot** retrieve hidden test cases via any endpoint; only ADMIN can create/publish; pagination works on a seeded set of a few hundred problems; `EXPLAIN ANALYZE` on the list query shows an index being used for the common filter (§7).

Stage 2 — Submission Lifecycle (execution stubbed)

Goal: get the *state machine and persistence* right before touching real execution. This de-risks the hard part by separating "modeling a submission" from "running code."

- ☐ Remodel `SubmissionEntity`: primary link to **problem** (+ optional battle later), `language` on the submission, `status` (QUEUED/RUNNING/COMPLETED/FAILED), `verdict`, per-metric fields (exec time, memory, failing test index, compile output), timestamps (B6).
- ☐ Expand `Verdict` to include `TIME_LIMIT_EXCEEDED`, `MEMORY_LIMIT_EXCEEDED`, `INTERNAL_ERROR`, plus lifecycle states or a separate `status` enum.
- ☐ Submit endpoint: validate, persist as QUEUED, return immediately with a submission id (async-shaped even though the "judge" is still a stub that assigns a fake verdict).
- ☐ Poll endpoint: get submission status/result by id (ownership-checked).
- ☐ **Idempotency & rate limiting**: prevent duplicate/rapid submissions (§9).

Verify: submitting returns instantly with a QUEUED id; a stubbed background process transitions it to COMPLETED with a mock verdict; a user can only read *their own* submissions; hammering submit is rate-limited.

Stage 3 — The Judge (real execution) — *the hard core*

Goal: replace the stub with real, isolated, resource-limited execution. This is where the project earns its rating. Treat §9 and §14 as the detailed spec.

- ☐ Stand up a **separate worker process** that pulls jobs, executes code in an **isolated sandbox** (container-based; strongly consider integrating **Judge0** or the `isolate` tool rather than rolling your own), enforces **CPU time, wall-clock, memory, process count, no-network, read-only FS** limits, and returns a structured result.
- ☐ Define the **job contract** (what the API enqueues) and **result contract** (what the worker returns) as explicit DTOs/schemas — this is your internal API between app and fleet.
- ☐ Handle every failure mode: compile error, runtime error, TLE, MLE, worker crash, timeout, poison messages (dead-letter), retries with idempotency.
- ☐ Per-test-case evaluation with correct verdict aggregation and first-failing-test reporting.

Verify: a deliberately infinite-loop submission returns TLE (not a hung worker); a fork-bomb / `while(true){malloc}` is contained and returns MLE/limit error without harming the host; killing

a worker mid-job re-queues or fails the job cleanly (no lost/duplicated verdicts); network access from inside the sandbox is impossible.

Stage 4 — Solo Practice Complete

Goal: the full single-player loop is polished and production-shaped.

- ☐ Submission **history** per user/problem with keyset pagination; per-user solved/attempted stats; per-problem acceptance rate.
- ☐ Problem search/filter/sort at real data sizes; verify indexes.
- ☐ Caching where measurement justifies it (problem statements, aggregate stats) — §15.

Verify: all list endpoints are paginated and index-backed; N+1 queries are eliminated on list endpoints (verify via SQL logs / statistics — §8).

Stage 5 — Battles (async first, real-time later)

Goal: correct competitive mechanics over plain request/response *before* adding sockets.

- ☐ Redesign `BattleEntity`: a real **state machine** (WAITING → IN_PROGRESS → FINISHED/ABANDONED), participants (model as a join table, not hardcoded `user1`/`user2`, so 1vN is possible later), the problem(s), per-participant submissions, scores, winner, start/end times.
- ☐ **Matchmaking / join**: create battle, join by code or simple queue; concurrency-safe seat filling (§7 locking).
- ☐ Scoring & winner determination with correct **concurrency** (two players finishing near-simultaneously must not both "win"; server is authoritative — never trust client-reported times).
- ☐ Match history.

Verify: simulate two clients submitting within milliseconds — exactly one correct winner is recorded, no double-scoring, under concurrent load; abandoned/disconnected battles resolve deterministically.

Stage 6 — Rating & Leaderboards

Goal: fair, consistent rating and cheap leaderboard reads.

- ☐ Implement a real rating algorithm (**Elo** to start; **Glicko-2** when you want to handle rating reliability/inactivity) as a **pure, unit-tested function**.
- ☐ Apply rating changes **transactionally** with battle finalization (rating update and battle result commit together, with optimistic locking on the profile — §7/§8).
- ☐ `RatingHistoryEntity` for auditability and graphs.
- ☐ Leaderboard reads that don't scan the whole table on every request (materialized view or Redis sorted set — §11).

Verify: rating math has unit tests against known Elo examples; concurrent battle finalizations don't corrupt ratings (optimistic-lock retry works); leaderboard read is O(log n)/cached, proven with `EXPLAIN ANALYZE` or a Redis ZSET.

Stage 7 — Real-Time Layer

Goal: live battle experience on top of already-correct battle logic.

- ☐ WebSockets (Spring WebSocket + STOMP) for battle rooms: opponent progress, timer, verdict push.
- ☐ Design for **horizontal scaling from the start**: WebSocket servers are stateless; shared state and fan-out go through **Redis pub/sub** so two players on different instances stay in sync (§12).
- ☐ Graceful reconnect, presence, and "source of truth is the DB/Redis, not the socket."

Verify: two browser tabs in one battle see each other's status live; killing/restarting one API instance doesn't desync the battle; a dropped socket reconnects and re-syncs from authoritative state.

Stage 8 — Production Hardening *(partly continuous from Stage 0)*

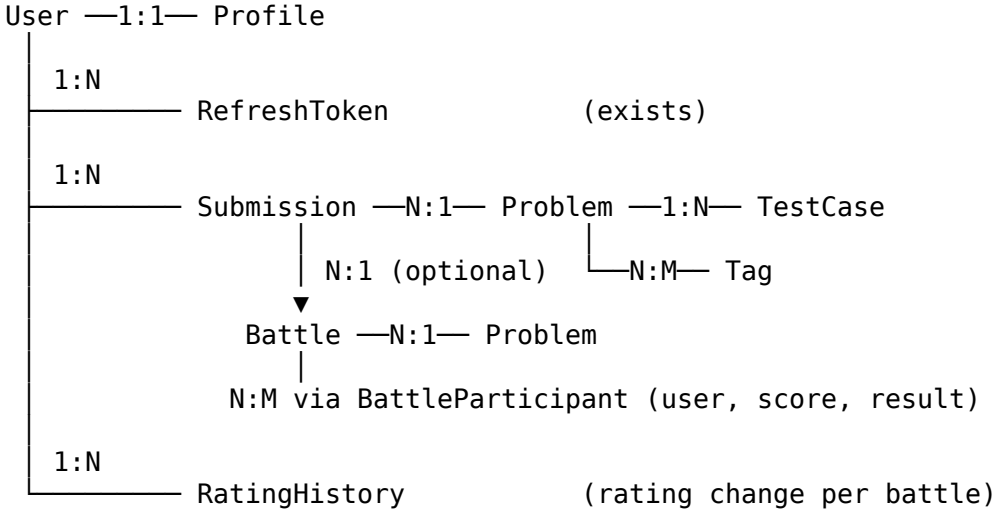
Goal: operable, observable, deployable. Details in §16/§17. Much of this you'll sprinkle in earlier; this stage is where you close the gaps: metrics + dashboards, tracing, alerting, backups & restore drills, CI/CD, autoscaling the worker fleet, load testing contest spikes.

---

6. Entity / Domain Roadmap

This is the conceptual data model to grow into — **not all at once.** Build entities as their stage arrives (§5). For each, I give the purpose, key relationships, and the design decisions that separate a robust model from a naive one. Field lists are conceptual guidance for you to implement, not schemas to copy.

6.1 The domain map (target state)



6.2 Entity-by-entity guidance

User (exists — refine). Authentication identity. Keep it lean: credentials, role, status (active/banned for moderation later). Move display/statistics concerns to Profile. Decision to make: stop having the entity implement `UserDetails` (B3); keep `CustomUserDetails` as the Spring Security adapter. Enforce `username`/`email` uniqueness in the DB (B2).

Profile (exists — refine). Public-facing identity + denormalized stats. Two things to fix: (1) it currently duplicates `userName` — decide whether the display name lives on User or Profile, not both. (2) `userWins`/`userLosses`/`xp` are **denormalized counters**; that's a legitimate performance choice (you don't want to `COUNT` battles on every profile view), but denormalized counters require **disciplined, transactional updates** and are a classic source of drift. Learn about this trade-off explicitly (§7 denormalization). Add a `rating` field here or in a dedicated rating entity (Stage 6). Create the profile **at signup** (your current signup doesn't) so the 1:1 invariant always holds.

Problem (exists — redesign in Stage 1). The authored content. Needs: title, unique slug, statement (markdown), difficulty, **time limit (ms)** and **memory limit (MB)** — these are *per-problem judge parameters*, `status` (DRAFT/PUBLISHED), author (→ User), timestamps. **Remove `language`** (B7). Free-text `constraints`/`complexities` are fine as display fields but aren't structured data.

TestCase (new — Stage 1). The heart of judging. Belongs to a Problem (N:1). Fields: input, expected output, `isSample` (samples are public; the rest are **hidden** and must never leak through any API), display order, optional weight/points. Design decisions: large inputs/outputs may not belong inline in Postgres forever (object storage later); a `@Lob`/TEXT column is fine for MVP. Cascade + orphan removal from Problem is appropriate here (deleting a problem deletes its test cases) — this is one of the few places `orphanRemoval=true` is correct (§8).

Tag (new — Stage 1). Category/topic. Many-to-many with Problem via a join table. Keep it simple: a tag is basically a name. Don't over-model taxonomies early.

Submission (exists — redesign in Stage 2). An attempt. Primary relationship to **Problem**; **optional** relationship to Battle (B6). Fields: author (→ User), language (enum, on the submission not the problem), source code (`@Lob`), `status` (lifecycle), `verdict` (outcome), execution time, memory used, index of first failing test, compile/stderr output (truncated), submittedAt/judgedAt. This entity is **append-only and high-volume** — it's the table that will dominate your row count, so it's the one you'll partition/archive first (§15).

Battle (exists — redesign in Stage 5). A competition instance. Needs a **status state machine** (WAITING/IN_PROGRESS/FINISHED/ABANDONED), the problem(s), start/end times, and a winner. **Do not hardcode user1/user2** — model participants as a separate entity so you can support 1vN, spectators, and per-participant scoring later without a migration.

BattleParticipant (new — Stage 5). Join entity between Battle and User carrying per-player state: score, finish time, result (WIN/LOSS/DRAW/ABANDONED), the submission that won it. This is where you enforce "one winner" logic transactionally.

RatingHistory (new — Stage 6). Append-only record of each rating change (user, battle, old rating, new rating, delta, timestamp). Enables rating graphs and auditability, and lets you recompute if your algorithm changes.

AuditLog / Notification / Report (much later). Admin/moderation and social. Deliberately deferred — see §19 on scope. Don't model these until Stages 0–6 are solid.

6.3 Cross-cutting entity decisions to make once

- **Timestamps:** you currently mix manual `@PrePersist` and Hibernate `@CreationTimestamp`. Standardize on **JPA auditing** (`@CreatedDate`/`@LastModifiedDate` with `@EntityListeners`) so every entity gets consistent, automatic timestamps.
- **IDs:** `IDENTITY` is fine, but understand that it forces a DB round-trip per insert and disables JDBC batch inserts. For high-volume inserts (submissions) a `SEQUENCE` with pooled allocation batches better. Learn the difference; don't prematurely switch everything.
- **Optimistic locking:** add a `@Version` field to any entity with concurrent mutable state — Profile (stats), Battle (state), rating. Not needed on append-only tables.
- **Enums as strings** (you do this) — keep it; it's migration-safe. Give enums an explicit, stable set of values.

---

7. Database Roadmap

You've started with PostgreSQL and indexing — good. Here's the progression of concepts, each tied to a concrete Code Brawl query so you learn it *in context*, plus how to *prove* your optimizations work. Do not "just add an index" — an index you can't justify is write-amplification you can't see.

7.1 Integrity first (Stage 0–1)

- **Foreign keys & constraints:** every relationship gets a real FK with an explicit `ON DELETE` policy you *chose* (restrict vs cascade). Decide per relationship: deleting a Problem cascading to TestCases = yes; deleting a User cascading to Submissions = probably no (archive instead).
- **Unique constraints:** `users.username`, `users.email` (B2), `problems.slug`, and any natural key. **This is a correctness feature, not an optimization** — it closes the signup race condition that application-level `existsBy...` checks cannot.
- **NOT NULL and CHECK constraints:** encode invariants in the schema (e.g., `time_limit_ms > 0`, rating within a sane band). The database is your last line of defense; use it.
- **Data integrity mindset:** the app can have bugs; the schema should make invalid states *impossible to store*, not merely *unlikely*.

7.2 Indexing — with justification (Stage 1 onward)

Learn to reason about indexes from the **query**, not the column. An index is justified when a *frequent* query filters/sorts/joins on those columns and the table is large enough that a sequential scan hurts.

Concrete Code Brawl cases:

Query	Index to consider	Why
”List published problems filtered by difficulty, newest first”	composite `(status, difficulty, created_at DESC)`	Matches the filter + sort; a composite index’s **column order matters** (equality columns first, range/sort last).
”A user’s submission history, newest first”	composite `(user_id, created_at DESC)`	The single most common submission read; without it you scan the biggest table.
”Submissions for a problem by verdict”	`(problem_id, verdict)` or a **partial index** `WHERE verdict='ACCEPTED'`	Partial indexes are smaller/faster when you only ever query one slice.
”Leaderboard by rating”	`(rating DESC)` — or don’t; use a ZSET/materialized view	Ranking over a whole table is the case where an index *isn’t enough* (§11).
Login	`users.username` unique index	Uniqueness + lookup in one.

Prove it or it didn’t happen. Learn `EXPLAIN` and especially `EXPLAIN (ANALYZE, BUFFERS)`:

- Look for **`Index Scan`/`Index Only Scan`** vs **`Seq Scan`** on your filter.
- Compare **estimated vs actual rows** — a big mismatch means stale statistics (`ANALYZE`) or a bad index.
- Watch **`Buffers`** (pages read) and total time; an ”index” that still reads most of the table isn’t helping.
- Understand that Postgres may *correctly* choose a seq scan on a small table — the planner is cost-based. Test against realistic data volumes (seed 100k+ submissions), because index behavior at 100 rows tells you nothing.

7.3 Pagination (Stage 1 onward)

- **Start with `OFFSET`/`LIMIT`** (what Spring Data `Pageable` gives you) — fine for early pages.
- **Understand why it degrades:** `OFFSET 100000` still scans and discards 100k rows. On deep pages (submission history of a heavy user, leaderboards) it’s slow.
- **Graduate to keyset / cursor pagination** (”give me the 20 after this id/timestamp”) for the big append-only tables. This is the production pattern and it plays perfectly with the `(user_id, created_at)` index above.

7.4 Transactions, isolation, locking, concurrency (Stage 5–6, but learn earlier)

This is where Code Brawl gets genuinely interesting, because battles and rating are concurrent.

- **Transactions:** one business operation = one transaction (on the service method). Know what ACID actually guarantees and what it doesn’t.
- **Isolation levels:** understand `READ COMMITTED` (Postgres default) vs `REPEATABLE READ` vs `SERIALIZABLE`, and the anomalies each prevents (dirty/non-repeatable/phantom reads, write skew). **Battle finalization is a write-skew risk**: two concurrent ”I finished first” transactions can each read ”no winner yet” and both claim victory. This is the canonical case for either `SERIALIZABLE` or explicit locking.
- **Locking:**
  - **Optimistic** (`@Version`) — best for low-contention mutable rows (Profile stats, rating). The update fails if someone changed the row; you retry. Cheap, no held locks.
  - **Pessimistic** (`SELECT ... FOR UPDATE`) — best for ”claim a seat in a battle” / ”exactly one winner”: you lock the row so only one transaction proceeds. Learn `FOR UPDATE` and lock ordering to avoid deadlocks.
- **Concurrency verification:** write a test that fires two threads at the same battle-finalize and asserts exactly one winner. Concurrency bugs don’t show up single-threaded.

7.5 Aggregation & denormalization (Stage 4–6)

- **Aggregation queries:** acceptance rate per problem, solved count per user, win/loss — learn `GROUP BY`, `COUNT`, `FILTER (WHERE ...)`, window functions (`RANK()`/`ROW_NUMBER()` for leaderboards).
- **When to denormalize:** your Profile already caches `wins/losses/xp`. That's denormalization — trading write complexity for read speed. Rule: **denormalize only when a measured read is too slow and the write path can keep the cache correct transactionally.** Otherwise compute on read.
- **Materialized views:** for leaderboards/dashboards that are expensive to compute but tolerate slight staleness — refresh on a schedule.

7.6 Migrations (Stage 0 — non-negotiable)

- Adopt **Flyway** (simplest) or Liquibase now. Every schema change is a versioned, reviewed, ordered migration file. `ddl-auto=validate` in all environments; Hibernate never alters schema.
- Learn **expand/contract (parallel-change) migrations** for zero-downtime changes later (add column → backfill → switch reads → drop old), because once you have data you can't just "update" a schema.

7.7 When caching enters (Stage 4+, only after measuring)

- Cache **read-heavy, rarely-changing** data: published problem statements, tag lists, leaderboard snapshots. Introduce Redis when a profiler/metric shows the DB is the bottleneck — **not before.**
- The hard part of caching is **invalidation**: decide TTL vs explicit eviction per cache, and know that a wrong invalidation strategy serves stale verdicts/ratings, which users notice immediately.

---

8. JPA / Hibernate Learning Roadmap

You're learning JPA while building — the right way to learn it. Below, each concept is tied to the exact place in Code Brawl where getting it wrong causes a real bug or performance cliff.

8.1 The mental model to build first

- **Persistence context / first-level cache:** the `EntityManager` tracks managed entities within a transaction; changes flush automatically (**dirty checking**) at commit. Understand *managed vs detached vs transient* states — most "why didn't my update save?" and "why did it save when I didn't call save?" confusion comes from not knowing which state an entity is in.
- **Entity lifecycle:** transient → managed (persist) → detached (tx ends) → removed. Your `@PrePersist`/`@PreUpdate` hooks are lifecycle callbacks; know when they fire.

8.2 Relationships & fetching — where your biggest performance bug lives

- **Default everything to `LAZY`.** `@ManyToOne` defaults to EAGER — that's a trap. Your `Submission → Battle`, `Submission → User`, `Battle → Problem` are all `@ManyToOne`; if left eager, loading a list of submissions eagerly drags in users, battles, problems.
- **N+1 queries — you *will* hit this at Stage 4.** Listing 20 submissions and touching `submission.getUser().getUsername()` for each fires 1 query for the list + 20 for the users = 21 queries. Learn to *detect* it (turn on SQL logging / statistics and count queries in a test) and *fix* it with:
  - **`JOIN FETCH`** in JPQL for a specific query,
  - **`@EntityGraph`** on a repository method for reusable fetch plans,
  - **batch fetching** (`hibernate.default_batch_fetch_size`) as a global safety net.

- **Cascades:** use deliberately. `CascadeType.ALL` from Problem → TestCase is reasonable; your current `User → Profile` `cascade=ALL` means deleting a user deletes the profile (fine) — but be sure you *want* cascade semantics; blanket `ALL` everywhere causes accidental deletes.
- **`orphanRemoval=true`:** correct for "remove a test case from a problem's collection = delete it." Wrong for associations where the child has independent life. Know the difference (cascade = propagate operations; orphanRemoval = delete children detached from the parent).

8.3 Reading data the production way

- **DTO projections for reads.** Don't load full entity graphs to render a list. Use interface- or constructor-based projections (or JPQL `SELECT new ...Dto(...)`) so the query fetches *only the columns you show*. This is the single biggest lever for list-endpoint performance and it sidesteps lazy-loading-during-serialization entirely.
- **JPQL vs Criteria vs native SQL:** JPQL for most queries; native SQL when you need Postgres-specific features (window functions for leaderboards, `INSERT ... ON CONFLICT`). Know that native queries bypass some JPA guarantees.
- **Pagination in Spring Data:** `Pageable` + `Page<T>`; understand it issues a separate `COUNT` query, and that `Page` (with count) vs `Slice` (no count) is a real performance choice for infinite-scroll vs numbered pages.

8.4 Transactions & locking in JPA (ties to §7.4)

- **`@Transactional` on service methods** = your transaction boundary and the scope of the persistence context. Learn propagation (`REQUIRED` vs `REQUIRES_NEW`) — e.g., writing an audit log that must persist even if the main tx rolls back needs `REQUIRES_NEW`.
- **Optimistic locking:** add `@Version` to Profile/Battle/rating; handle `OptimisticLockException` with a retry. This is how you make concurrent stat/rating updates safe *without* holding DB locks.
- **Pessimistic locking:** `@Lock(PESSIMISTIC_WRITE)` on the repository query for "claim battle seat" / "declare winner."

8.5 The one anti-pattern to disable now

- **Open Session In View (OSIV)** is *on* by default in Spring Boot. It keeps the persistence context open during view rendering, which *hides* N+1 problems and holds DB connections longer than needed. **Set `spring.jpa.open-in-view=false`** and fix the lazy-loading exceptions it surfaces properly (with fetch joins / projections). Doing this early forces good habits; doing it late means untangling a codebase that quietly relied on it.

8.6 Entity design discipline

- **Entities are for persistence; DTOs are for API.** Never serialize entities to JSON (leaks, lazy blow-ups, coupling). You already do this for auth — universalize it.
- **`equals`/`hashCode` on entities:** don't use Lombok `@Data`/`@EqualsAndHashCode` blindly on entities — it can trigger lazy loads and break with generated IDs. Learn the recommended approach (business key or ID-based with care). You currently use `@Getter`/`@Setter`/`@Builder` which is safer than `@Data` — good instinct; just know *why*.

---

9. Submission & Code-Execution Architecture

This is the defining system of Code Brawl. I'll frame it as the three tiers you asked for, then call out the concurrency/failure concerns explicitly. **Security of untrusted execution is in §14 — read both together.**

9.1 The submission lifecycle (get this right regardless of tier)

Model submission processing as an explicit **state machine**, because "a submission" is a long-lived thing that moves through stages and can fail at each:

```
RECEIVED → QUEUED → COMPILING → RUNNING → JUDGED (verdict)
              │                     │
              ▼                     ▼
        (rejected /          (COMPILE_ERROR / RUNTIME_ERROR /
         rate-limited)        TLE / MLE / INTERNAL_ERROR)
```

The API's job is to accept, persist as QUEUED, and return **immediately** with an id. The worker's job is to move it through the rest and write the terminal result. The client polls (MVP) or is pushed to (real-time later). **The request thread must never wait for code to run** — that's the cardinal rule; violating it means one slow submission ties up a web thread and a contest spike takes the whole API down.

9.2 Tiered evolution

Concern	**MVP** (learn the shape)	**Solid** (real judge)	**Production-grade**
Async mechanism	In-app async (a @Async method or a DB-polled "QUEUED" worker thread)	Real message broker (**Redis Streams / RabbitMQ**); API is a producer, workers are consumers	Partitioned/priority queues (contest vs practice), backpressure, autoscaling consumers
Where code runs	A **separate worker process**, executing inside a **Docker container** per submission with limits — or integrate **Judge0** and treat it as your execution engine	Dedicated worker fleet; containers built per language with pinned toolchains; **`isolate`**-style resource control	Pooled/pre-warmed sandboxes, gVisor/Firecracker microVM isolation, multi-tenant fairness
Isolation	Container: non-root user, `--network none`, read-only FS, dropped capabilities, CPU/memory/pids limits	+ seccomp profile, no host mounts, ephemeral throwaway containers	+ kernel-level isolation (gVisor/Firecracker), per-tenant quotas
Limits enforced	Wall-clock timeout (kill container), memory limit, output size cap	+ CPU-time (not just wall), process/thread count, file size, compile timeout	+ fine-grained cgroup accounting, deterministic CPU pinning for fair timing
Failure handling	Timeout → TLE; crash → mark FAILED	Retries with idempotency, **dead-letter queue** for poison jobs, worker heartbeat/visibility timeout	Circuit breakers, partial-result handling, automatic worker replacement
Results	Single verdict	Per-test verdicts, first failing test, time/memory per run, truncated stderr	Streamed progress, result caching for identical (problem, code-hash)

Strong recommendation for your MVP: do **not** write your own sandbox first. Either (a) run a container-per-submission with the standard hardening flags and let the worker orchestrate it, or (b) stand up **Judge0** (open-source, self-hostable) and make your worker a thin client to it. Your learning value is in the **orchestration, lifecycle, queueing, and failure handling** — which are 100% yours to build — not in re-deriving Linux sandboxing, which is a security minefield (§14).

9.3 Concurrency & correctness concerns (explicitly)

- **Duplicate submissions / double-submit:** debounce on the client is not enough. Enforce server-side: a short-window idempotency key, and/or reject a new submission while the user has a QUEUED/RUNNING one for the same problem. Otherwise a double-click doubles your judge load.
- **Idempotent job processing:** design so that if a job is delivered twice (queues guarantee *at-least-once*, not *exactly-once*), the worker doesn't write two verdicts. Key the result write on submission id + a processing token.
- **Poison messages:** a submission that crashes the worker must not crash it forever in a loop — after N attempts it goes to a dead-letter queue and is marked INTERNAL_ERROR.
- **Fair scheduling:** during a contest, one user submitting 50 times shouldn't starve everyone else. This is why priority/partitioned queues appear at the production tier.
- **Determinism:** the same submission should get the same verdict. Wall-clock timing is noisy under load; production judges pin CPU and measure CPU-time. Know this exists even if MVP tolerates noise.

9.4 What to keep simple initially

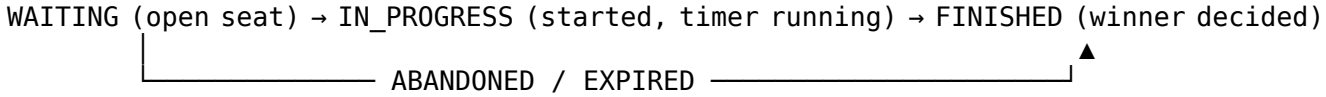
- One language end-to-end first (pick Java or Python), *then* generalize the language→toolchain mapping. Don't build a six-language matrix before one works.
- Store test cases inline in Postgres for MVP; move to object storage only when sizes demand it.
- Poll for results in MVP; add push (WebSocket) only in Stage 7.
- Skip result caching, pre-warmed pools, and microVMs until you've measured a need.

---

10. Competitive Battle Architecture

A battle is, at its core, **a race: N players solving the same problem(s), scored by correctness + speed, with an authoritative server deciding the outcome.** Build the *logic* correctly over request/response first; add real-time (§12) only once the logic is right.

10.1 The battle state machine



Every transition is a guarded, transactional operation. Illegal transitions (submit to a FINISHED battle, join a full battle) must be rejected by the service, not just hidden by the UI.

10.2 Tiered evolution

Concern	**MVP**	**Solid**	**Production-grade**
Matchmaking	Create + join by battle code (private rooms)	Simple queue: pair the two longest-waiting compatible players	Rating-based matchmaking, wait-time widening, regional pools
Participants	`BattleParticipant` join entity (avoid hardcoded user1/user2)	+ spectators, reconnection, ready-check	+ N-player free-for-alls, teams
Scoring	First correct verdict wins	Points by test coverage + time + penalties for wrong attempts	Configurable scoring rules per battle mode
Concurrency	`SELECT FOR UPDATE` on battle when declaring winner	Optimistic version + retry; idempotent finalize	Distributed coordination if battle state ever spans instances
State location	Postgres (poll)	Postgres + Redis for live counters/timers	Redis-authoritative live state, DB for durable record

10.3 The concurrency problem that defines this system

Two players submit correct solutions 5ms apart. Both worker results come back. Two threads run "finalize battle." **Without protection, both read `winner == null` and both set themselves as winner** — you've recorded two winners (write skew). This is *the* teaching moment of the battle system:

- Serialize finalization with a **pessimistic lock** on the battle row (`FOR UPDATE`), or use `SERIALIZABLE` isolation, or an atomic conditional update ("set winner = me **where** winner is null").
- **Never trust client-reported finish times.** The server timestamps when it *received and judged* the submission. A cheating client will lie about its time; the server is the only clock that counts.
- Write a test that spawns concurrent finalizers and asserts exactly one winner. If you take one idea from this section, take that test.

10.4 Keep simple initially

Async (submit → poll) battles first. No sockets, no matchmaking algorithm, no spectators. A battle you can join by code and win by submitting first, with correct concurrency, is a *complete* Stage 5. Everything else is polish.

---

11. Ranking System

Rating and leaderboards look simple and are full of subtle traps: fairness, consistency under concurrency, and the surprisingly expensive ”what’s my rank?” query.

11.1 Rating algorithm

- **Start with Elo.** It’s simple, well-documented, and correct enough: expected score from the rating difference, update by `K * (actual − expected)`. Implement it as a **pure function** (inputs: two ratings + outcome; output: new ratings) so it’s trivially unit-testable against known examples.
- **Graduate to Glicko-2** when you want to model **rating reliability (RD)** and **inactivity** — it answers ”how confident are we in this rating?” which Elo can’t. It’s more math but a great learning step.
- **Keep `RatingHistory`** (append-only) so you can draw graphs and, critically, **recompute** if you change algorithms. Never destroy the audit trail by only storing the current number.

11.2 Consistency

Rating changes must commit **in the same transaction** as the battle result, with optimistic locking (`@Version`) on the profile/rating row. If two battles for the same user finalize concurrently, the version check forces a retry so you don’t lose an update (lost-update anomaly). This is §7.4/§8.4 applied.

11.3 Leaderboards — the query that doesn’t scale the obvious way

”Top 100 by rating” is easy. **”What is *my* rank (position) among 500k users?”** is the expensive one — naively it’s a full ordering on every request.

Tier	Approach
MVP	`ORDER BY rating DESC LIMIT/OFFSET`, index on rating. Fine for small data.
Solid	**Materialized view** refreshed on a schedule for the ranked board; window functions (`RANK()`) computed periodically, not per request.
Production	**Redis sorted set (ZSET):** `ZADD` on rating change, `ZREVRANK` gives a user’s rank in `O(log n)`, `ZREVRANGE` gives top-N instantly. The DB stays the durable source of truth; Redis is the fast index.

Learn *why* the ZSET wins: it maintains order incrementally on write, so reads never re-sort. That’s the general lesson — for hot ranking/counting, maintain the answer on write instead of computing it on read.

11.4 Keep simple initially

One global rating, one leaderboard, Elo, computed with a plain query. Add per-language/per-category boards, seasons, and Redis only when data volume or product need forces it.

---

12. Real-Time Architecture

Real-time is a **UX layer over already-correct battle logic** (§10). If battles are wrong via HTTP, sockets just make them wrong faster. Build this in Stage 7, not before.

12.1 Transport choice

- **Server-Sent Events (SSE)** if you only need server→client push (live opponent progress, timer, verdict) — simpler, works over plain HTTP, auto-reconnect built in.
- **WebSockets (Spring WebSocket + STOMP)** if you need bidirectional low-latency (typing presence, chat, live cursors). More capable, more complexity. For Code Brawl's battle room, WebSocket/STOMP is the likely fit.

12.2 The scaling trap to design around from the first line

Two players in one battle may connect to **different API instances** behind a load balancer. In-memory socket state on one instance is invisible to the other, so player A's progress never reaches player B.

- **Keep WebSocket servers stateless.** The authoritative battle state lives in Redis/DB, not in a server's memory.
- **Use a shared pub/sub (Redis) as the message backplane:** an event published on instance 1 fans out to subscribers on instance 2. Spring's STOMP broker relay / a Redis-backed broker does this. Design for it *now* even if you run one instance in dev — retrofitting a backplane after assuming single-instance memory is a rewrite.
- **Sticky sessions** are a crutch, not a solution; prefer stateless + backplane.

12.3 Correctness under disconnects

- The socket is a *delivery mechanism*, never the source of truth. On reconnect, the client re-syncs from authoritative state (DB/Redis), it doesn't replay missed socket frames blindly.
- Handle presence/absence explicitly: a dropped connection ≠ abandoned battle; decide timeouts.

12.4 Keep simple initially

Polling is a legitimate MVP for live-ish updates (poll submission/battle status every second or two). Ship battles with polling, then upgrade the hot paths to push. Don't build a socket infrastructure before you have battles worth watching.

---

13. Testing Strategy

Test the **risk**, not the framework. You don't need 100% coverage; you need high coverage on the parts where a bug is expensive (money, security, correctness) and just enough elsewhere. Follow the **pyramid**: many fast unit tests, fewer integration tests, a handful of end-to-end tests.

Test type	What to test in Code Brawl	Tooling / notes
Unit	Pure business logic with no Spring context: **rating math (Elo/Glicko)**, verdict aggregation, battle scoring rules, state-machine transition guards, keyset-pagination cursor logic.	Plain JUnit + AssertJ; mock collaborators with Mockito. These should run in milliseconds and be the bulk of your tests.
Repository / persistence	Custom queries, projections, cascades, constraints, that your indexes exist, that unique constraints actually reject dupes, that migrations apply.	`@DataJpaTest` **with Testcontainers PostgreSQL** — see the H2 warning below.
Web / controller	Request→response mapping, validation rejections, status codes, that entities never leak, that the response envelope is consistent.	`@WebMvcTest` + MockMvc for slice tests; service mocked.
Security	**The most under-tested and highest-value area here.** Can a USER hit an admin endpoint? Can user A read user B's submissions? Are hidden test cases unreachable? Does an expired/forged token get rejected? Does RBAC actually work after you fix B1?	`spring-security-test` (`@WithMockUser`, `SecurityMockMvcRequestPostProcessors`). Write an explicit "authorization matrix" test.

Test type	What to test in Code Brawl	Tooling / notes
Business-logic / concurrency	The two-players-finish-simultaneously test (one winner). Concurrent rating updates (no lost update). Duplicate-submission rejection.	Multi-threaded tests / `CountDownLatch`; assert invariants hold under contention.
Integration / E2E	The full loop: submit → job enqueued → worker judges → verdict persisted → retrievable. Battle create → join → both submit → correct winner + rating change.	`@SpringBootTest` with Testcontainers (Postgres + Redis + a real worker/Judge0 container). Few but high-value.

Critical warning — do not test persistence on H2. H2 is not PostgreSQL: it emulates a different SQL dialect, handles types/UPSERT/JSON/locking/indexes differently, and will happily pass tests that then fail in production. **Use Testcontainers to run the real PostgreSQL image** in tests. This is exactly why Stage 0 stands up Testcontainers before you have much to test.

What deserves the most tests: the judge pipeline, the battle finalization concurrency, the authorization matrix, and the rating math. A bug in any of those is either a security hole, a cheating vector, or a "the leaderboard is wrong" credibility killer.

What barely needs tests: trivial getters, straight-through CRUD with no logic, framework behavior you didn't write. Don't test that Spring Data's `save()` works.

---

14. Security Strategy

Security here spans the ordinary web app concerns **and** the extraordinary one — running hostile code. I'll separate them. (Per your request, this covers *architecture and principles* for untrusted execution, not a recipe for building or escaping a sandbox.)

14.1 The web-app layer (you have auth — this is what to verify/add)

- **Authentication:** you have JWT + refresh tokens. Verify: separate secrets (B5), sane lifetimes (B4), refresh tokens **stored hashed** in the DB (so a DB leak doesn't hand out live sessions — yours are currently stored raw), rotation on refresh, and revocation on logout/all-devices.
- **Authorization — two levels, and the second is the one everyone forgets:**
  - *Role-level (RBAC):* `@PreAuthorize("hasRole('ADMIN')")` — but only after fixing B1, or it silently fails.
  - *Object-level (ownership):* "is this submission's `user_id` the caller's id?" RBAC says a USER may read submissions; **ownership** says *which* submissions. Missing ownership checks are the #1 real-world API vulnerability (IDOR). Every "get by id" for user-scoped data needs one.
- **Sessions & cookies:** you set the refresh token in an `HttpOnly` cookie (good) but with `secure(false)` — must be `true` in production (HTTPS-only), plus `SameSite`. Decide your token transport deliberately: access token in `Authorization` header, refresh token in `HttpOnly` cookie is a reasonable model — but see CSRF.
- **CSRF:** you disabled it. That's correct **for stateless, header-based JWT**. But the moment a browser sends the refresh token from a cookie automatically, the `/refresh` endpoint is CSRF-relevant. Mitigate with `SameSite` and consider CSRF protection specifically on cookie-driven state-changing endpoints. Understand *why* the rule "disable CSRF for JWT" has this exception.
- **CORS:** not configured yet. You'll need an explicit allow-list of frontend origins, methods, and headers (never * with credentials). Configure it centrally.
- **Input validation:** Bean Validation on every request DTO (you do this for auth — universalize). Validate sizes hard: **cap submitted source length**, cap request bodies, cap test-case sizes.
- **Rate limiting & abuse prevention:** protect login (brute force), signup (bot accounts), and **submissions** (judge is expensive — this is your cost/DoS surface). Start with a per-user/IP limiter (Bucket4j or a gateway/Redis token bucket).

- **Secret management:** environment variables now (12-factor), a real secret store (Vault, cloud secrets manager) later. Never in `application.properties` (C1). Rotate the already-leaked password.
- **Security logging/auditing:** log auth events (login success/failure, token refresh, role changes, admin actions) — **without logging the tokens themselves** (B8). This is your audit trail for moderation and incident response.

14.2 The untrusted-code execution layer (the hard part — principles only)

The governing principle: **assume every submission is actively malicious and will try to exfiltrate data, consume all resources, escape its environment, and attack other users' runs.** Your job is to make the blast radius of a fully-compromised execution as small as possible.

Defense-in-depth layers to understand (each is a ring around the code):

1. **Never run user code in the API process or with app privileges.** Physical/process separation from your application and database is non-negotiable — a compromised run must not be able to reach your Postgres, your secrets, or your network.
2. **Isolation boundary.** Containers are the *baseline*; stronger isolation (user-space kernels like **gVisor**, or microVMs like **Firecracker**) exists precisely because container escapes are a real category. Know the spectrum and where you sit.
3. **Resource limits** enforced by the platform, not the code: CPU time *and* wall-clock, memory, process/thread count (stops fork bombs), file descriptors, output size, disk. A submission that hits a limit gets a clean verdict (TLE/MLE), not a hung host.
4. **No network.** Deny all egress from the sandbox. Competitive submissions never need the internet; allowing it invites data exfiltration and outbound attacks.
5. **No persistence / ephemerality.** Each run gets a fresh, read-only-where-possible, throw-away environment. Nothing a submission writes survives or is visible to the next run — this prevents cross-submission attacks and information leaks.
6. **Least privilege.** Non-root user, dropped capabilities, restricted syscalls (seccomp). The process can do arithmetic and I/O to its judged streams and essentially nothing else.
7. **Don't build this yourself first.** Use **Judge0** or the IOI **`isolate`** tool, which encode years of hard-won hardening. Your engineering effort belongs in orchestration, limits configuration, and failure handling — not in re-inventing sandboxing, where a subtle mistake is a full host compromise.

Verification mindset: treat "can a submission read the host filesystem / reach the network / exceed limits / affect another run / crash the worker permanently" as explicit test cases and red-team your own judge before you trust it.

---

15. Performance & Scalability Strategy

Optimize in this order: **correct → measured → fast.** Never optimize what you haven't measured; you'll spend effort on the wrong thing and add complexity you can't justify.

15.1 Know your workload shapes

- **Read-heavy, cache-friendly:** problem lists/statements, leaderboards, profiles. Scale with pagination, projections, and (later) caching.
- **Write-heavy, bursty:** submissions during a contest — thousands arrive in seconds. Scale with the **queue as a shock absorber** (accept fast, process as capacity allows) and **horizontally scalable workers**.
- **Stateful, latency-sensitive:** live battles. Scale with stateless servers + Redis backplane (§12).

15.2 The likely bottlenecks, in the order you'll hit them

1. **N+1 queries** on list endpoints (Stage 4) — fix with projections/fetch joins (§8), not hardware.
2. **The database connection pool** — understand HikariCP sizing; a pool that's too big overwhelms Postgres, too small starves the app. Measure.

3. **Deep pagination** on submissions/leaderboards — keyset pagination (§7.3).
4. **Leaderboard ranking** — ZSET/materialized view (§11).
5. **Judge throughput** — the real scaling story: workers are CPU-bound and independently scalable; add workers, add priority queues, autoscale on queue depth.
6. **Only then**, caching (Redis) for hot reads — with a deliberate invalidation strategy.

15.3 Scalability principles to internalize

- **Keep the app tier stateless** so you can run N instances behind a load balancer. Push all shared state to Postgres/Redis. (Your JWT-stateless design already helps here.)
- **Decouple with the queue** so producers (API) and consumers (judge) scale independently and a spike degrades latency, not availability.
- **Backpressure over collapse:** when the judge is saturated, submissions wait in the queue with a visible status; the API stays responsive. Design the UX around ”queued” being normal.
- **Load-test the contest spike** before you claim it scales. A synthetic burst of submissions is the test that matters.

15.4 Keep simple initially

One app instance, one or few workers, no cache, 0FFSET pagination, plain leaderboard query. Add each scaling mechanism only when a metric shows you need it. Premature scaling is as harmful as premature optimization.

---

16. Production-Readiness Checklist

Split by ”now” (cheap, do during Stage 0 and as you go) vs ”later” (when you have real traffic or are actually deploying). ”Now” items make development itself saner.

Concern	**Now (Stage 0 → as you build)**	**Later (approaching deployment / scale)**
Logging	Structured logs, correlation id per request, no secrets/tokens (fix B8), sensible levels (auth → DEBUG)	Centralized aggregation (ELK/Loki), retention, PII scrubbing
Error handling	One global handler, `ProblemDetail` (RFC 7807), never leak stack traces to clients, map every custom exception	Error budgets, user-facing error catalog
API docs	springdoc/Swagger UI generated from code	Published, versioned API reference for frontend
Migrations	Flyway from day one, `ddl-auto=validate`	Expand/contract zero-downtime migrations
Config	Externalized via env vars, per-profile (`dev`/`test`/`prod`), 12-factor	Config service / typed config validation on startup
Secrets	Out of git, env vars, rotate leaked ones	Vault / cloud secret manager, rotation policy
Health checks	Actuator `health`/`info`, DB + Redis health indicators	Kubernetes liveness/readiness probes wired to them
Metrics	Micrometer counters/timers on key ops (submissions/sec, judge latency, queue depth)	Prometheus + Grafana dashboards, SLOs
Monitoring/alerting	—	Alerts on queue depth, error rate, judge failures, DB saturation
Tracing	—	Distributed tracing (OpenTelemetry) across API → queue → worker
Rate limiting	Basic per-user/IP on login/signup/submit	Distributed (Redis) limits, per-plan quotas
Backups	Know your `pg_dump`; don’t lose the dev DB	Automated backups + **restore drills** (a backup you’ve never restored isn’t a backup), PITR
Failure handling	Retries + dead-letter for judge jobs, timeouts on all external calls	Circuit breakers, graceful degradation, chaos testing
Observability	Enough logs/metrics to answer ”why is this submission stuck?”	Full traces + dashboards to answer it in seconds

---

17. Deployment / Infrastructure Roadmap

Grow this in tiers too; don't jump to Kubernetes on day one.

- **Local / dev:** `docker-compose` running Postgres + Redis + your app + a worker. Fix the current `compose.yaml`/properties mismatch (C3) so it's one reproducible command. This is also what your Testcontainers tests mirror.
- **Containerize properly:** multi-stage Docker build (build with Maven, run on a slim JRE), non-root container user, externalized config via env, a `.dockerignore`. The **worker is its own image** with the language toolchains + isolation tooling.
- **CI/CD (add early, it pays for itself):** on every push — build, run unit + Testcontainers integration tests, run a security/dependency scan, build images. GitHub Actions is the natural fit. Fail the build on test/scan failure. This is how you keep the growing codebase honest.
- **First real deployment:** a single VM or a small managed platform (Railway/Render/Fly/managed container service) with a **managed Postgres** (so backups/PITR aren't your problem yet) and a managed Redis. Put the app behind HTTPS (TLS termination) with `secure` cookies.
- **Scaling the fleet:** run the judge workers as a separately scalable service; autoscale on queue depth. This is where container orchestration (Kubernetes/Nomad, or a managed equivalent) earns its complexity — **only when a single box can't keep up.**
- **Environment configuration:** distinct dev/`staging`/`prod` configs, secrets injected per environment, no shared credentials. Database migrations run as a deploy step, not by the app on boot (in prod).

---

18. Things to Learn Along the Way

Grouped by when they become load-bearing, so you learn just-in-time instead of all-at-once.

Before Stage 1 (data + fundamentals): relational modeling & normalization; FKs, unique/CHECK constraints; Flyway migrations; Spring Data pagination & projections; `EXPLAIN ANALYZE` basics; Bean Validation; `open-in-view=false` and why.

Before Stage 3 (the judge): message queues (delivery guarantees: at-least-once vs exactly-once, idempotency, dead-letter); async processing in Spring; container fundamentals (images, resource limits, namespaces at a conceptual level); the isolation spectrum (containers → gVisor → microVMs); the security model of running untrusted code; Judge0 / `isolate` as reference systems.

Before Stage 5–6 (competition + rating): transaction isolation levels & the anomalies they prevent; optimistic vs pessimistic locking; write-skew and the "two winners" problem; Elo, then Glicko-2; window functions & aggregation in Postgres.

Before Stage 7 (real-time): WebSockets/STOMP vs SSE; stateless scaling + pub/sub backplane; Redis data structures (especially sorted sets); reconnection/presence patterns.

Continuous (all stages): the testing pyramid & Testcontainers; observability (logs/metrics/traces); 12-factor config & secrets; HikariCP/connection-pool tuning; API design (versioning, RFC 7807, idempotency); reading `EXPLAIN` output fluently.

Meta-skill: get comfortable **measuring before optimizing** — SQL logging, query counts in tests, Micrometer timers, and load tests. The senior habit is "prove the bottleneck," not "guess the bottleneck."

---

19. Common Architectural Mistakes to Avoid

Ordered roughly by how badly they'll hurt you here.

1. **Executing untrusted code without real isolation** (or building your own sandbox first). This is the catastrophic one — a mistake means host/DB/secret compromise. Use a proven engine; assume malice.
2. **Trusting the client** for scores, finish times, verdicts, or ownership. The server is the only authority. This is both a correctness and an anti-cheat requirement.

3. **Enforcing integrity in application code instead of the database.** Your signup uniqueness race (B2) is exactly this. Constraints belong in the schema.
4. **Blocking a web thread on code execution.** Async from the start (§9). Synchronous judging takes the whole API down under load.
5. **Missing object-level authorization** (IDOR). RBAC ≠ ownership. Every user-scoped `get by id` needs an ownership check.
6. **Exposing entities over the API.** Leaks fields (password hashes!), triggers lazy-loading serialization errors, and welds your API contract to your schema. Always DTO.
7. **Relying on `ddl-auto=update`.** Unreviewable, unrollbackable schema drift. Migrations from day one.
8. **Leaving OSIV on** and letting it hide N+1 problems until they surface as production latency.
9. **`OFFSET` pagination on your biggest tables** (submissions, leaderboards). Keyset instead.
10. **Computing leaderboard rank on every request.** Maintain it on write (ZSET/materialized view).
11. **No idempotency on judge jobs** → duplicate/lost verdicts when the queue redelivers.
12. **Premature everything:** microservices, Kubernetes, caching, six languages, real-time — before the core is correct. Complexity you can't justify with a metric is a liability.
13. **Package-by-layer forever**, until every service can touch every repository and boundaries dissolve. Move to package-by-feature.
14. **Testing persistence on H2.** It lies. Testcontainers + real Postgres.
15. **Secrets in git** (C1). Rotate and externalize.
16. **Scope creep into breadth** (notifications, friends, OAuth, AI, gRPC, moderation) before depth (problems → judge → battles → rating) is solid. This is the mistake most likely to make the project *feel* stalled.

---

20. Clear Immediate Next Steps

You asked, explicitly, what to work on **right now** so you don't scatter. Here it is, in order. Resist starting anything below the line until the above is done.

Do these now, in this order

1. **Rotate the leaked DB password and pull all secrets out of `application.properties`** into environment variables. (Highest priority — it's a live exposure in git.) *(C1)*
2. **Introduce Flyway, set `ddl-auto=validate`,** and capture the current schema as migration #1. From here, the database is versioned. *(C2)*
3. **Fix the RBAC wiring (B1)** — the double `ROLE_` prefix — and prove it with a throwaway admin-only endpoint test. Your authorization is currently mis-wired; nothing role-based will work until this is right.
4. **Add DB-level `UNIQUE` constraints** on `username` and `email`, and consolidate to a single `UserDetails` implementation. *(B2, B3)*
5. **Stand up Testcontainers + PostgreSQL** and write one green repository integration test. This unlocks trustworthy testing for everything after.
6. **Standardize the API contract:** one response envelope, `/api/v1` prefix, add Swagger. Do this while the API is tiny. *(C6)*
7. **Clean house:** remove unused gRPC/Shell/AI deps, delete `TempTask`, stop logging tokens, set `open-in-view=false` and fix whatever breaks. *(C4, B8)*

That is **Stage 0** — a focused foundation-hardening pass. It's mostly small, it's all cheap right now, and it makes every later stage safer. None of it is "rebuilding auth"; it's fixing specific defects and laying the cross-cutting bones.

Then, the first real feature

8. **Build the Problem domain (Stage 1):** `ProblemEntity` (redesigned — no `language`), `TestCaseEntity`, `TagEntity`, admin authoring + publishing, and public paginated/filtered browsing that never leaks hidden test cases. **Everything else — submissions, the judge, battles, rating — depends on problems and test cases existing.** This is the highest-leverage feature to build next.

The one-sentence focus

> **Harden the foundation (Stage 0), then build Problems + Test Cases (Stage 1), then pour your real energy into the judge (Stage 3).** Problems → real judge → battles → rating, done well, in that order. Treat notifications, friends, OAuth, AI, and real-time as *later* — they are the sirens that sink this kind of project.

End of roadmap. Revisit and revise this document as you complete each stage — it's meant to be a living source of truth, not a monument.